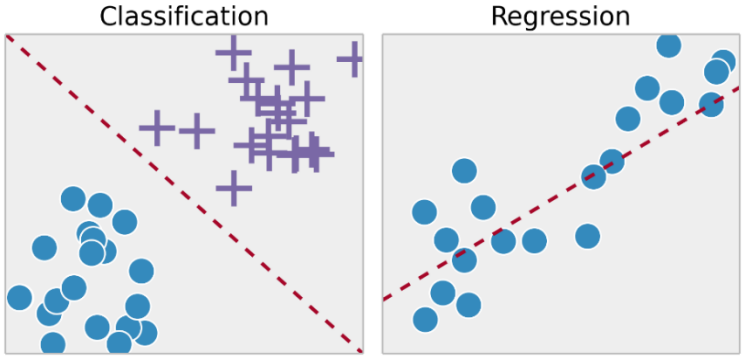
IE540
L1. Introduction to DP & RL

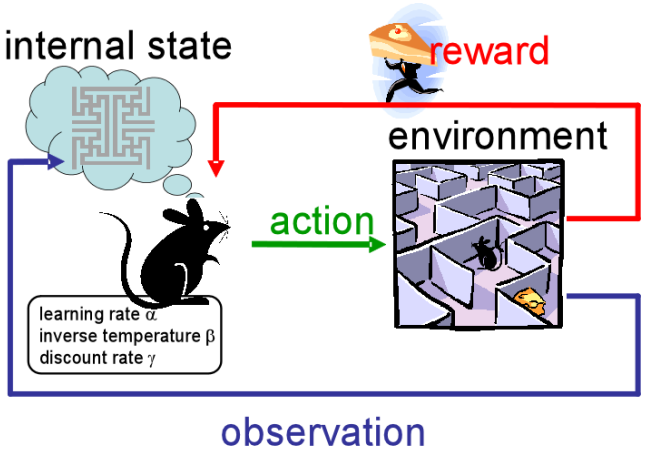
KAIST ISE
Hayong Shin

Types of Machine Learning

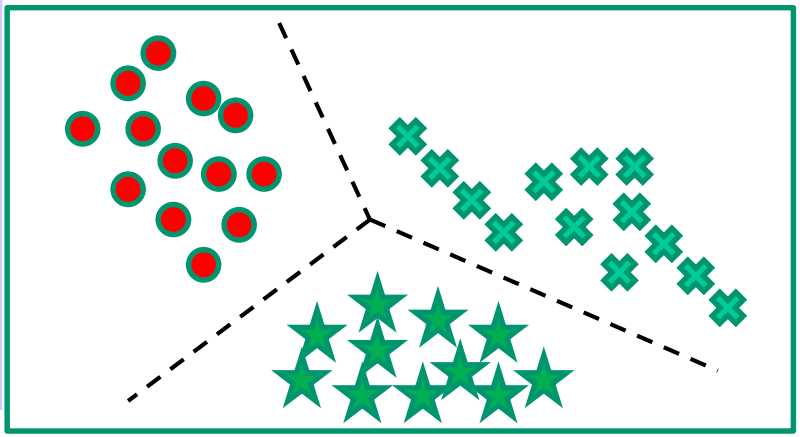
- **Supervised learning**
  - Training data : Input Feature + Answer $\{(\boldsymbol{x}_i, y_i); i = 1..N\}$
    - $\boldsymbol{x}_i$: input (feature) vector
    - y_i : label
  - Regression (y_i : cardinal values)
  - Classification (y_i :
  - …



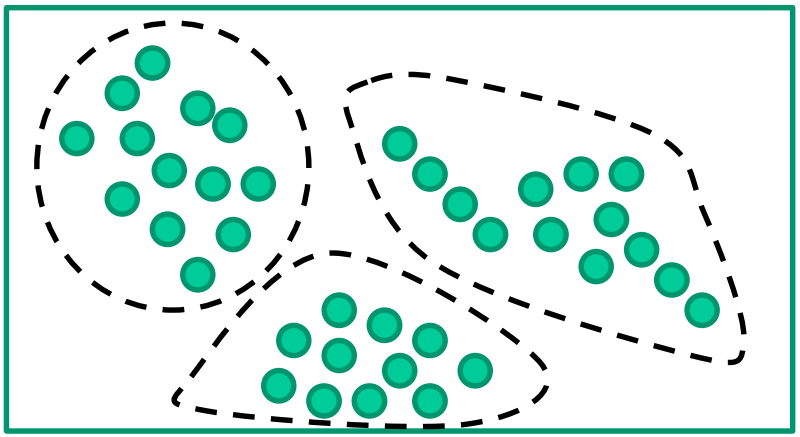
- **Unsupervised learning**
  - Training data : No labels $\{\boldsymbol{x}_i; i = 1..N\}$
  - Clustering, Probability distribution estimation, Dimension reduction, …
- **Semi-supervised learning**
  - Labeled data + Unlabeled data
- **Reinforcement learning ??**
  - Learning from



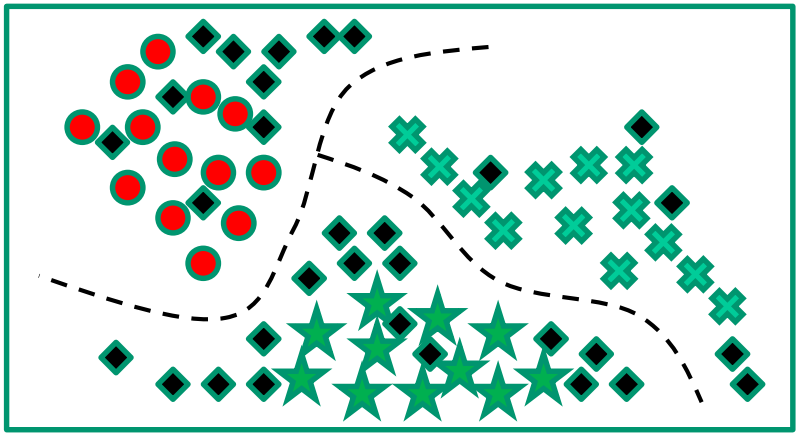
Types of Machine Learning



Supervised learning



Unsupervised learning



Semi-supervised learning

Reinforcement learning



The Road Not Taken

BY ROBERT FROST

Two roads diverged in a yellow wood,
And sorry I could not travel both
And be one traveler, long I stood
And looked down one as far as I could
To where it bent in the undergrowth;

Then took the other, as just as fair,
And having perhaps the better claim,
Because it was grassy and wanted wear;
Though as for that the passing there
Had worn them really about the same,

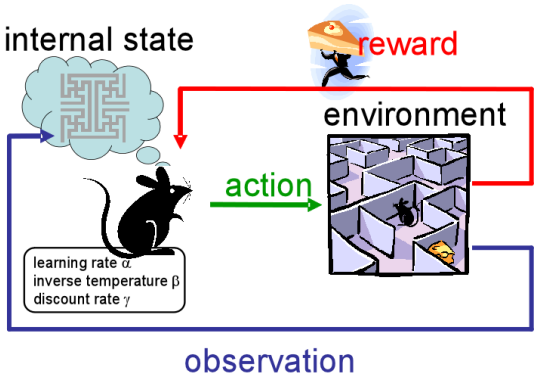
And both that morning equally lay
In leaves no step had trodden black.
Oh, I kept the first for another day!
Yet knowing how way leads on to way,
I doubted if I should ever come back.

I shall be telling this with a sigh
Somewhere ages and ages hence:
Two roads diverged in a wood, and I—
I took the one less traveled by,
And that has made all the difference.



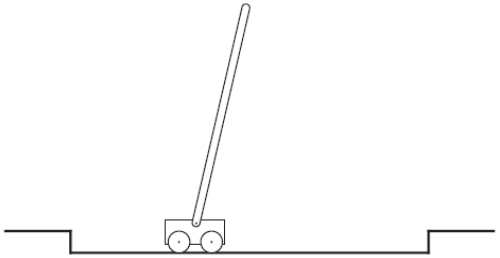
Characteristics of RL

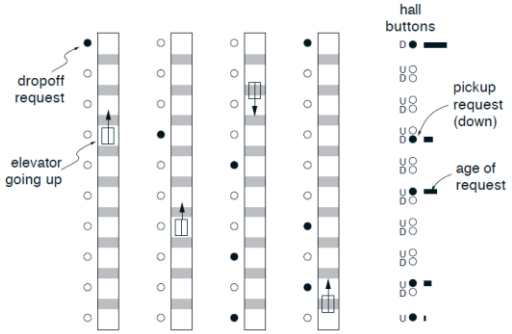
- Supervised learning
  - to learn $y = f(x)$ from data $\{(\boldsymbol{x}_i, y_i); i = 1..N\}$
    - $\boldsymbol{x}_i$: input (feature) vector
    - y_i : label
- Reinforcement learning
  - to learn a policy : state → action
  - no supervisor
    - reward signal : feedback from environment
    - max expected return (immediate + future) : value function
      - reward is delayed
  - at the beginning
    - agent can interact with environment
      - state → action → consequence (reward, next state)
    - agent's actions affect the subsequent data trajectory
    - censored data : consequence for taken actions only
    - exploitation vs exploration tradeoff

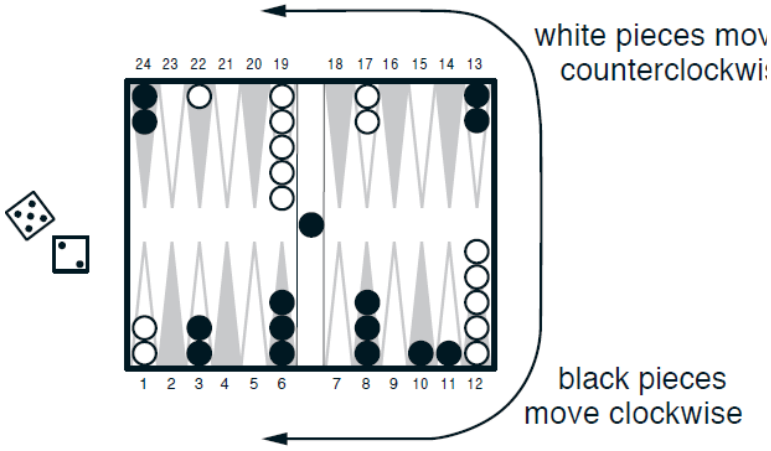


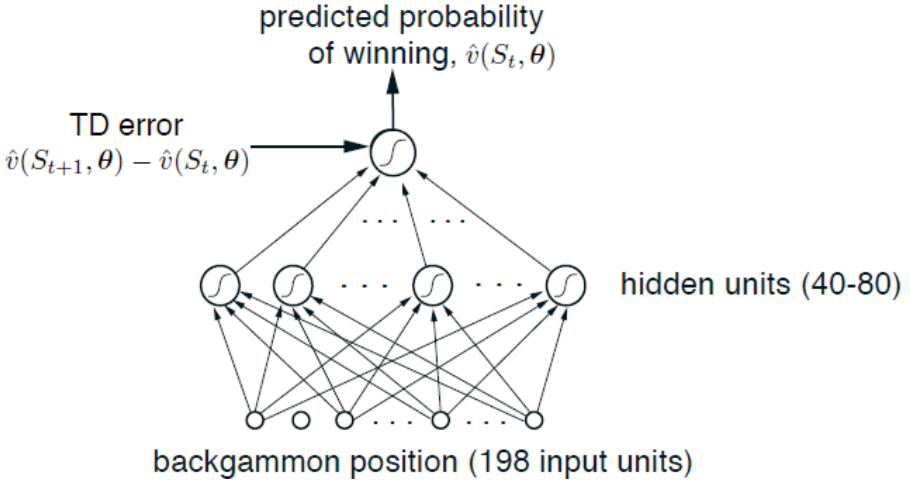
RL applications

- RL has been successfully applied in many tasks
  - Pole balancing
  - Drone flipping
  - Backgammon : RL + NN, beaten human players
  - Elevator dispatching
  - Dynamic channel allocation in cellular network
  - Job shop scheduling
  - …
- Relatively simpler tasks

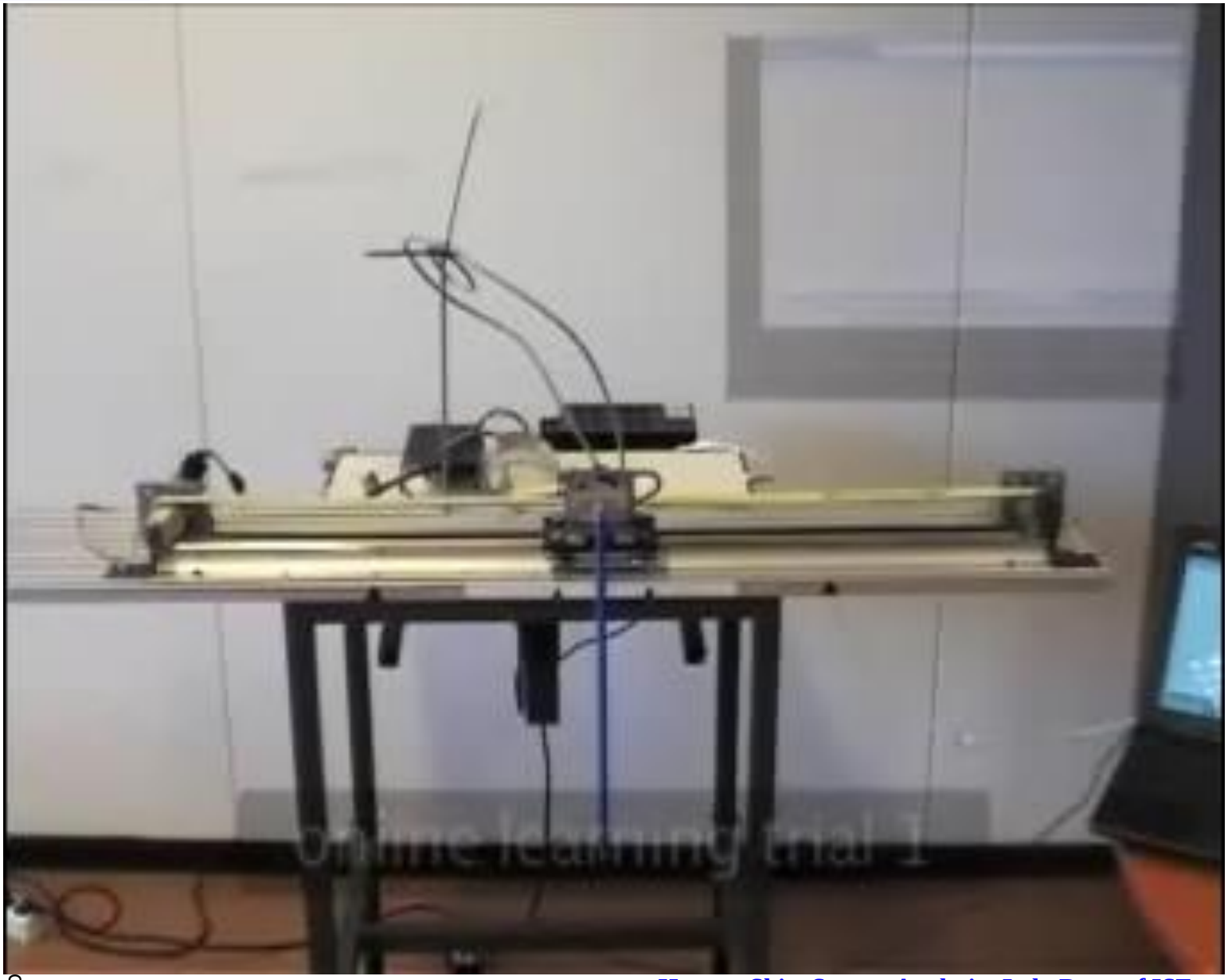








Reinforcement learning - pole balancing



Google Deepmind

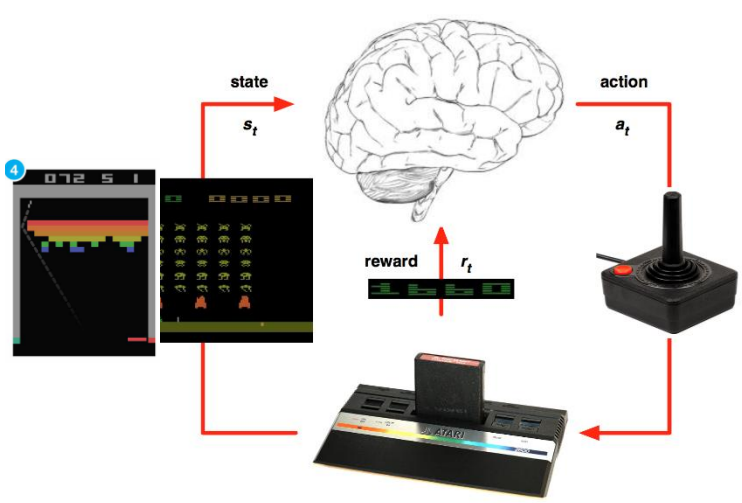
- In search of AGI : Artificial General Intelligence, like human
  - Instead of Narrow AI (hand-crafted software only for specific task)
    - Deep Blue, which defeated human champion of chess
- 2015 Nature paper : Playing 49 Atari video games like human

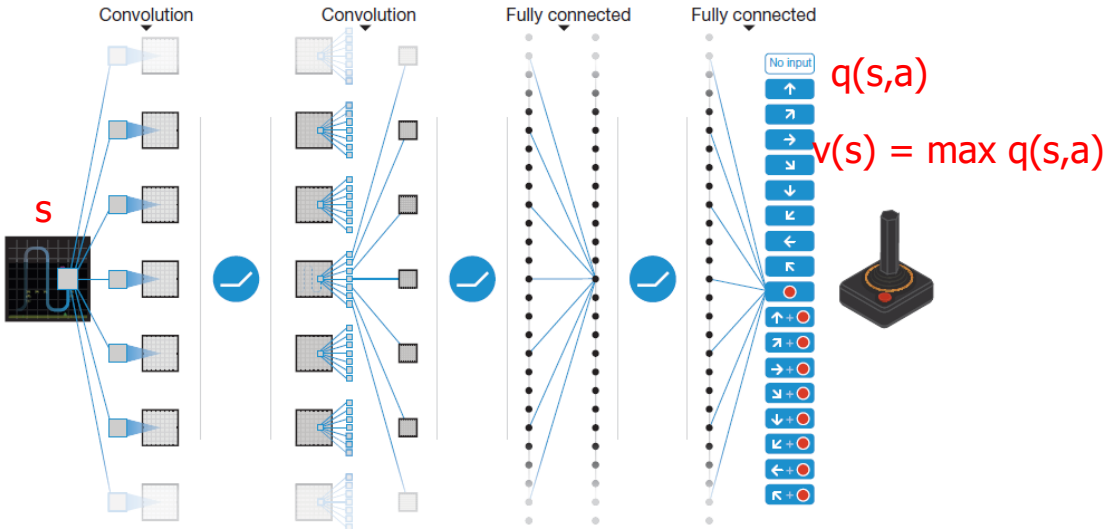
  * **Reinforcement learning** framework

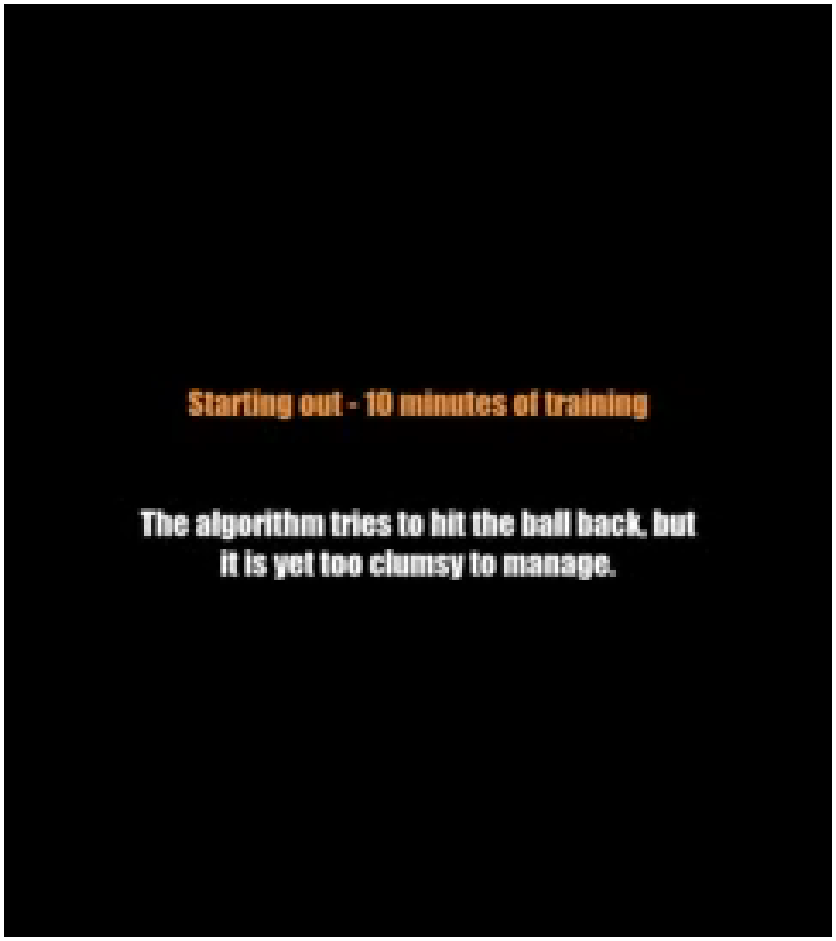
  + **Deep learning** for approximating value function

  - Without any prior knowledge about the games

  → Huge success in almost all games





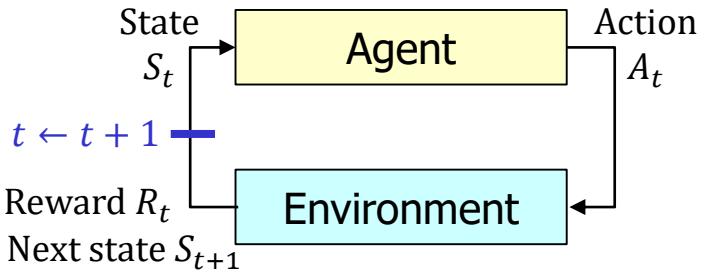

Starting out - 10 minutes of training
The algorithm tries to hit the ball back, but
it is yet too clumsy to manage.


AlphaGo
Lee Sedol

Dynamic system, Sequential decision making

- Dynamic system
  - a system whose state S_t changes over time t
    - eg. go game : board configuration
  - cf. static system
    - no notion of state or time
    - eg. roulette : no state

- Sequential decision making
  - system behavior depends on agent's decision
    - behavior : state change + reward
  - System = Environment + Agent
  - Environment : passive
  - Agent : active
    - selection action
    - to maximize total reward in the future



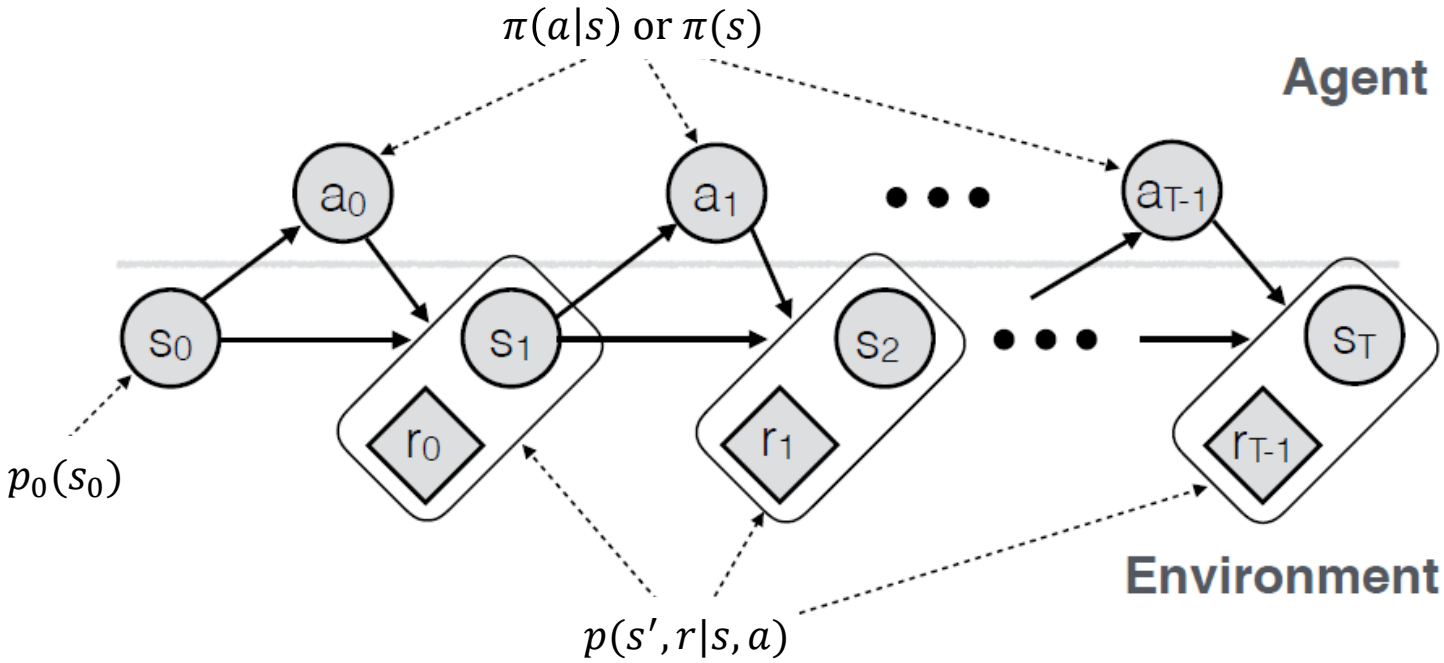
MDP vs DP vs RL

- MDP (Markov Decision Process)
  - Sequential decision making problem in a dynamic system
    - a.k.a. DP problem → confusing, not recommended
  - MDP : a formalism for seq. dec. making problem w/ Markov property
  - described by : state, state transition, action, reward, policy
  - Recursive formula (Bellman eq.)
- DP (Dynamic Programming) algorithm
  - DP for static problems (as an algorithmic pattern)
    - recursively defined problems
    - solve smaller problem → store result → use it for bigger problem
    - eg : string matching, knapsack, shortest path, …
  - DP for MDP problem : backward induction or fixed point iteration
    - solution method for MDP : VI, PI
    - curse of dimensionality : large state space, action space
    - curse of model : unknown state transition model, reward model
- RL (Reinforcement Learning) ≈ Approximate DP
  - Solve DP problem approximately without knowing the model
    - Learn from experience (interaction with the environment)
    - Use functional approximation to the solution

Rewards

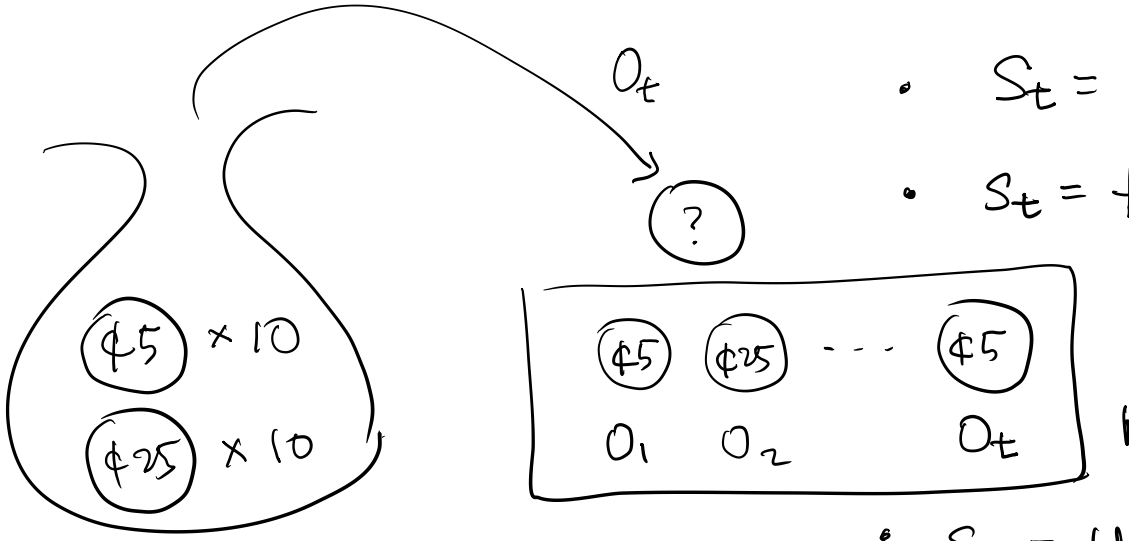
- reward R_t
  - scalar feedback signal

- RL is based on the following reward hypothesis
  - all goals of an agent can be described by the maximization of expected cumulative reward (= return)
  - TAQ : Do you agree with the reward hypothesis?



State

- State S_t
  - environment state S_t^{env}
  - observation O_t : information observed by agent
    - history $H_t = (O_0, A_0, R_0, O_1, A_1, R_1, \dots, O_t)$
  - agent state s_t^{agent} : state perceived by agent
    - full observation : $O_t = S_t^{env} = S_t^{agent}$
    - partial obs. : $O_t \neq S_t^{env}$, S_t^{agent}: agent's belief about S_t^{env}
- Markov state (=information state) assumption
  - $p(S_{t+1}|\boldsymbol{S}_{0:t}, A_t) = p(S_{t+1}|S_t, A_t)$
- State transition
  - deterministic transition : $S_{t+1} = f(S_t, A_t)$
  - Stochastic transition
    - functional form
      - $S_{t+1} = f(S_t, A_t) + \eta_t$: additive noise
      - $S_{t+1} = f(S_t, A_t, \eta_t)$, $\eta_t \sim p(.)$: noise
    - distributional form
      - $S_{t+1} \sim p(.|S_t, A_t)$



- $S_t = O_t$ Markov ?
- S_t = total sum of money. Markov ?
- $H_t = (O_1, O_2, \cdots, O_t)$
- $S_t = H_t$ Markov ?
- $S_t = (n_t, q_t)$ Markov ?

n_t : # of nickels out, $\bar{n}_t = 10 - n_t$: remaining nickels

q_t : # of quarters out, $\bar{q}_t = 10 - q_t$: " quarters

Taxonomy of Dynamic System

- Time horizon T (ending time)
  - Finite horizon : T is finite and known
    - current time t (= stage) matters
    - in state transition and reward at stage t : $p_t(s'|s,a)$, $R_t(s,a)$
    - decision also depends on the stage t : $\pi_t(s)$
  - No-horizon : T is unknown or infinite
    - indefinite horizon : T finite but unknown
      - positive probability of reaching terminal (=absorbing) state
    - infinite horizon : T is infinite → Continuing task
      - no absorbing (= terminal) state
    - stationarity for no-horizon (either indefinite or infinite)
      - stage t does not affect either system behavior or decision
      - stationary transition and stationary reward : $p(s'|s,a)$, $R(s,a)$
      - stationary decision : $\pi(s)$
        - » stage t does not matter
  - episodic task : task with finite or indefinite horizon
  - continuing task : infinite horizon task

Taxonomy of Dynamic System

- Discrete vs Continuous
  - State space
    - discrete state
      - eg. gridworld : cell index
    - continuous state
      - eg. cartpole : position, velocity, angle, angular velocity
  - Action space
    - discrete action
      - eg. cartpole: -1(left) or +1(right)
    - continuous action
      - eg. modified cartpole: force $\in$ [-1, +1]
  - Time
    - discrete time
    - continuous time

Major components of RL

- **model** : agent's representation of the environment
  - cf. simulation model
  - prediction about the reward R_t and the next state S_{t+1}
    - State transition model
      - $p^a_{s,s'} = p(s'|s,a)$
    - reward model
      - distribution $p(r|s,a)$
      - expected reward $R(s,a) = \mathbb{E}[R_t|S_t = s, A_t = a] = \mathbb{E}_{R\sim p(r|s,a)}[R]$
  - Joint model of state transition and reward
    - $p(s',r|s,a)$: next state and reward may be dependent
      - next state dist. $p(s'|s,a) = \sum_r p(s',r|s,a)$
      - reward dist. $p(r|s,a) = \sum_{s'} p(s',r|s,a)$
      - expected reward $R(s,a) = \mathbb{E}_{R\sim p(r|s,a)}[R] = \sum_{r,s'} p(s',r|s,a)\, r$
    - if $R(s,a,s') = \mathbb{E}_{R\sim p(r|s,a,s')}[R]$ is given $\left[p(r|s,a,s') = \frac{p(s',r|s,a)}{p(s'|s,a)}\right]$

$$R(s,a) = \mathbb{E}_{s'\sim p(s'|s,a)}[R(s,a,s')] = \sum_{s'} p(s'|s,a)R(s,a,s')$$

$$= \sum_{s'} p(s'|s,a) \sum_r r \frac{p(s',r|s,a)}{p(s'|s,a)} = \sum_r \sum_{s'} p(s',r|s,a)\, r = \sum_r p(r|s,a)r$$

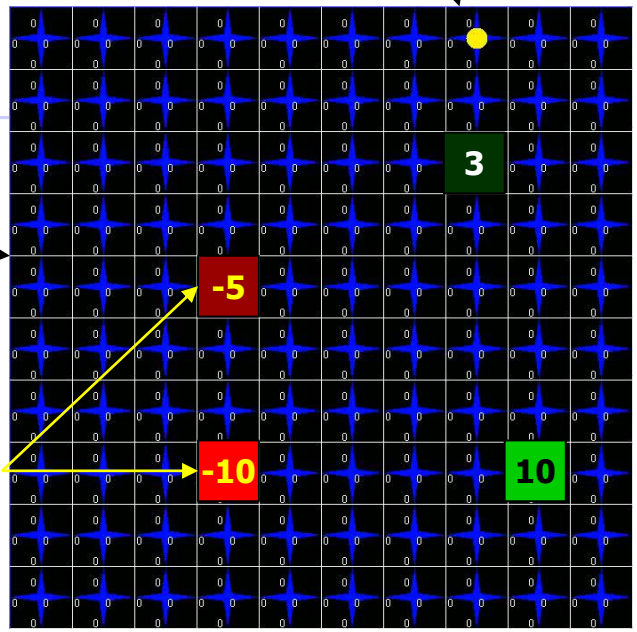
Major components of RL

- **policy** : agent's behavior function to choose action
  - deterministic policy : $A_t = \pi(S_t)$
  - stochastic policy : $A_t \sim \pi(a|S_t)$

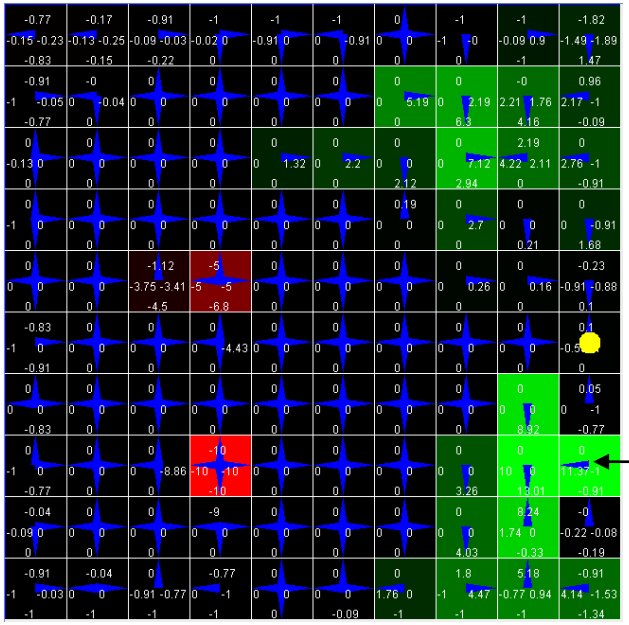
- **value function** : how good is a state S_t
  - expected return when following a policy π from a state S_t
  - state value function
    - $V^{\pi}(S_t) = \mathbb{E}_{A_{t:T}\sim\pi}[R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots | S_t]$
  - action value function :
    - $Q^{\pi}(S_t, A_t) = \mathbb{E}_{A_{t+1:T}\sim\pi}[R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots | S_t, A_t]$

State s

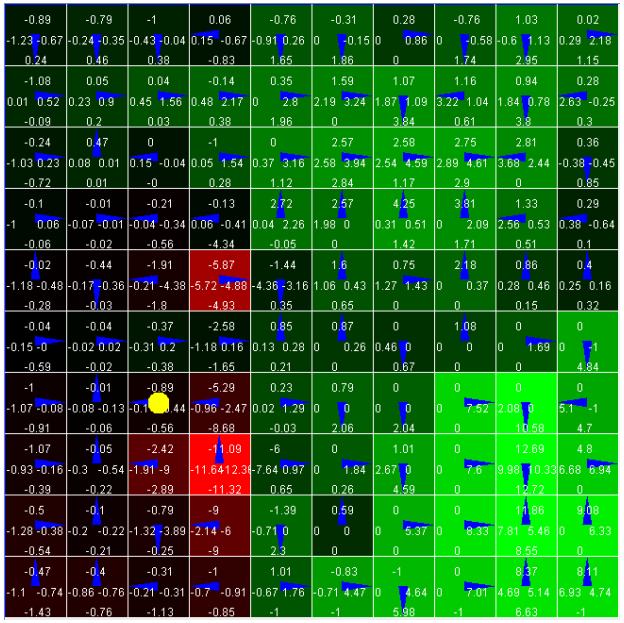
Value Function V(s)

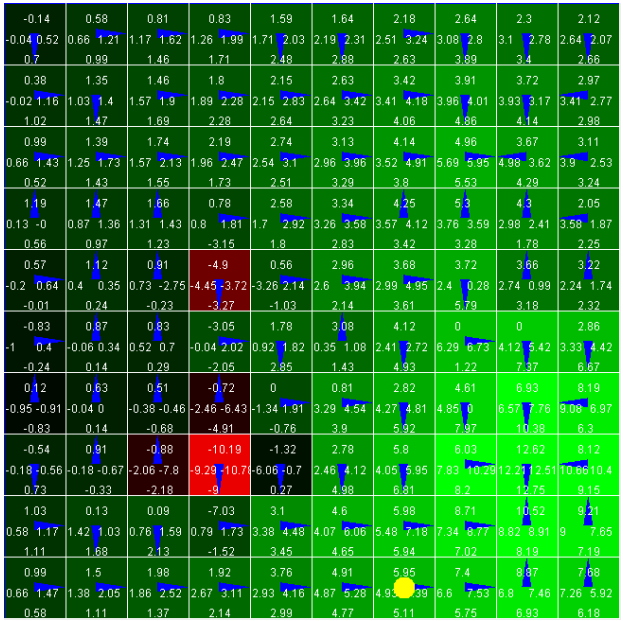


Reward

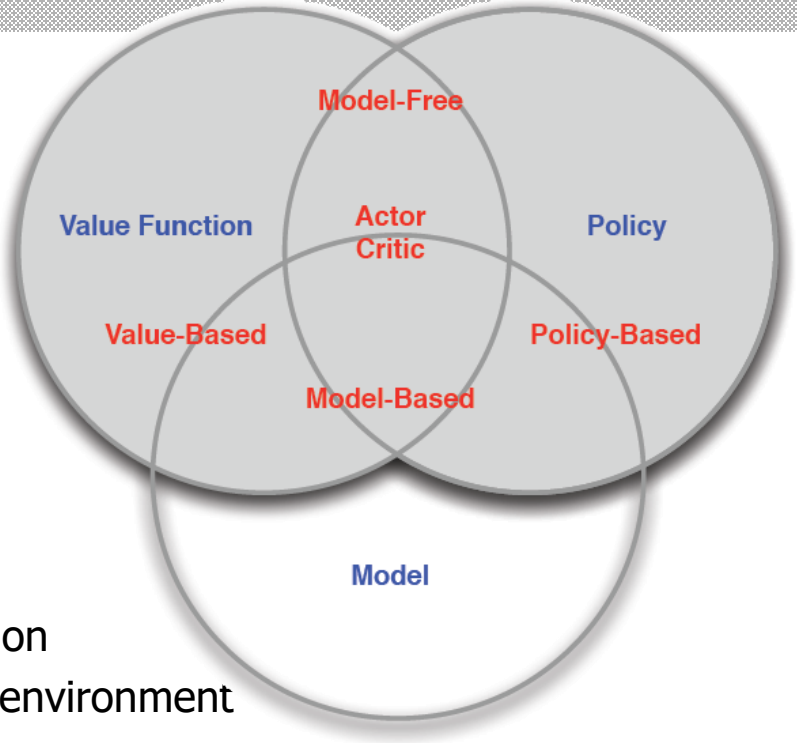


Policy Function π(s)



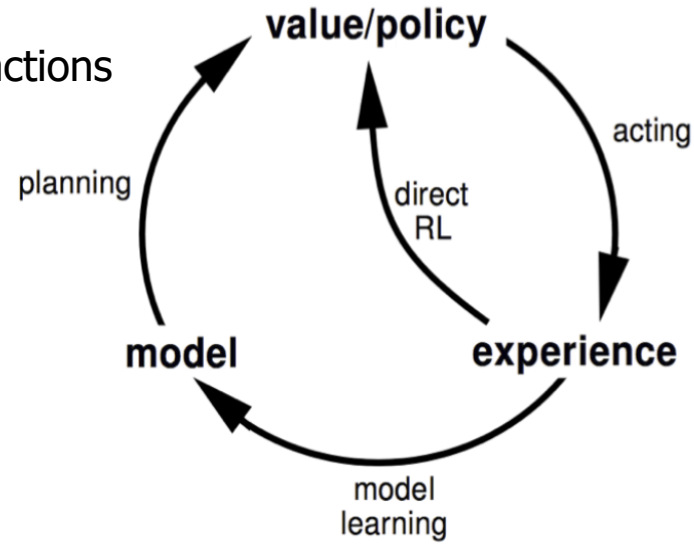


Categorizing RL agents



- Value vs Policy
  - value based
  - policy based
  - value + policy : actor-critic

- Model
  - 'model' : state transition + reward distribution
    - doesn't mean simulation model of real environment
  - model free : direct RL
    - no model
    - learn value and/or policy directly from interactions
  - model based
    - learn model from interactions
    - use model to learn value and/or policy
  - integrated approach
    - learn value/policy directly
    - learn model at the same time
    - use model to facilitate learning value/policy



Categorizing RL agents

- Online vs Offline
  - online learning : learn from real environment
    - reward during learning matters
    - exploitation only?
  - offline learning : learn from (simulated) environment (learnt policy matters)
    - reward during learning doesn't matter
    - obtaining better policy is important during training period
    - more exploration
    - TAQ. no exploitation for offline learning?
- On-policy vs Off-policy
  - on-policy : behavior policy = learning target policy
  - off-policy : behavior policy ≠ learning target policy
    - TAQ. is off-policy for online learning meaningful?